

Signals Associated with Page Visibility and a Ranker for Editorial Review

Zain Ul Ebad Khan

Abstract

I ask whether page-level signals, like measured search position, how many days a page had usable data, and word count, are associated with a page’s search visibility, and whether that association is strong enough to help rank pages for editorial review. I then built a page-month dataset from FlyRank’s internship warehouse, defined a top 20 percent visibility label from March 2026 impressions, and trained a logistic regression ranker against a holdout that groups pages by client so no client’s pages appear on both sides of the split. On that grouped holdout the ranker reached a precision at 20 of 0.85 and a precision at 50 of 0.64 against a base rate of 0.169252, and the inverted-position baseline scored 0.00 at both cutoffs on the same test set. A FlyRank editor takes that ranked list and decides, page by page, whether to review it or skip it, rather than treating any single score as a signal to publish or reject something automatically. These results are observational and meant for decision support, not causal evidence that any specific action changes visibility, and they are not a claim that I’ve recovered or reverse-engineered Google’s ranking algorithm.

1 Introduction

FlyRank’s editors can’t manually review every page a client publishes, so I wanted to know whether page-level signals could help point them toward the pages most likely to matter for search visibility. The unit of analysis in this paper is a single page in March 2026, identified by a client hash and a content hash. What I’m building toward is two things: signal notes describing how a page’s measured position, click-through rate, and word count relate to its visibility, and a ranked review list that orders pages by a model score. An editor’s action on that list is to review a page or skip it, and the cost of getting this wrong cuts both ways, as reviewing pages that were never going anywhere wastes editor time, while skipping a page that was actually gaining visibility means missing something real.

I compare the ranker against a simple baseline, inverted position (1 divided by average position), because it’s transparent and needs no training, and I hold both to the same client-grouped split inside each notebook so the comparison is fair. This paper is the hybrid I built after moving from the Week 1 starter CSV Ranking Signal Analysis into FlyRank’s warehouse, and it narrows that same question with a fuller dataset rather than being a second paper.

2 Data

The data for this paper comes from FlyRank’s internship warehouse, a gated Hugging Face dataset that I queried with DuckDB. I worked from three tables: `fact_content_daily_performance`, restricted to the March 2026 partition, along with `dim_content` and `dim_clients` for page and client attributes. The grain is one page, identified by a client hash and a content hash, for the month of March 2026, and uses 331,437 pages built from 9,841,378 page-days between March 1 and March 31, 2026. This is not the Week 1 starter CSV; it’s a page-day rollout from the warehouse itself.

I left out `fact_content_query_90d` because its grain is a rolling 90 day query window on a different clock than the March page-day table, and mixing the two would mean joining data measured over incompatible time spans. I also excluded a June-dated table with a `_sample` suffix for the same reason, and I never used the client or content hashes as model features, only as keys to group pages by client for the holdout split. I

also avoided coding unmeasured days as position zero: a page with no measured position on a given day is missing data, not evidence that it ranked at the bottom, so those days are excluded rather than zero-filled.

That choice still leaves real coverage gaps. Of the page-days in the March partition, 3,611,061 were measured and 6,230,317 were not, and 154,699 pages were never measured at all during the month.

3 Methodology

My assumptions going into this are modest. I assume a page’s average position, when it’s actually measured, is a reasonable stand-in for how prominently that page shows up in search results, and that a handful of slower-moving page attributes, like word count and how many days Google Search Console had usable data for a page, capture something about a page’s history rather than a snapshot of one lucky day. I also assume that a page with no measured position at all is missing information, not a page that ranked at the very bottom.

The features I put into the ranker are `avgPosition`, `measuredDayCount`, `positionSpread`, `wordCount`, and `gscHistoryDays`, plus a set of missing value flags for the same fields. I left out click-through rate, clicks, impressions, search trend data, and any identifying hashes, names, or URLs, because those either measure the same thing I’m trying to predict or would let the model memorize specific pages instead of learning a pattern. The label, `isTopVisibility`, marks a page as 1 if its March impressions fall in the top 20 percent among pages with a measured average position, with that cutoff computed only on the training half of whichever holdout arm I’m running, never on the full dataset.

The baseline I compare against is as follows: score equals 1 divided by average position when average position is at least 1, and 0 otherwise, tagged with the reason code `assoc.position` and the action `watch_in_brief`. It needs no training and no tuning. The model itself is a scikit-learn `LogisticRegression`, trained on the five features above.

I evaluate both the baseline and the model on two kinds of holdout. A grouped-by-client split puts every page from a given client entirely in either train or test, so the model never sees a client’s other pages during training and can’t quietly memorize a client specific pattern. A random split shuffles pages without regard to client, which lets pages from the same client land on both sides, and lets the model implicitly learn client identity rather than generalize. I only trust the grouped split as evidence of anything; the random split shows up once in this paper, and it’s labeled as leaky.

I also ran a few checks aimed at catching features that leak the label into itself. In an earlier association pass, a linear regression on the five candidate features alone scored a Spearman correlation of 0.8402 against March impressions, in-sample, on the 110,403 pages with complete data for all five. Adding click-through rate barely moved that number, to 0.8398, but adding the log of impressions jumped it to 0.8912, which is exactly what I’d expect from a feature that’s really just the label in disguise, so neither click-through rate nor log-impressions made it into the ranker. I treated `measuredDayCount` as a feature to watch rather than trust outright, too: it counts how many days a page had a usable measurement, which tracks the same underlying activity as the label rather than acting as an independent signal, and the Results section reports what happens to precision when I drop it. The client and content hashes were never used as model inputs; they only group pages by client for the split.

4 Results

The ranking numbers in this paper come from one capstone re-run against the warehouse (seed 42), not from averaging Week 5 and Week 6. Table 1 and Table 2 share the same grouped-by-client test set. I am not presenting them as two different client draws.

On that grouped split, 266,437 pages sat in train and 65,000 in test (44 train clients, 11 test clients, zero client overlap). The model scored 20,632 test pages that had a usable label. Precision at 20 was 0.85 and precision at 50 was 0.64, against a base rate of 0.169252, with a train-only impression cutoff of 1,611.0. Table 1 puts that headline next to a random-page-split run, labeled as a leaky comparison rather than a second result. Of 265,149 train rows and 66,288 test rows, 34,735 scored test pages, a cutoff of 1,563.0, precision 0.95 at 20 and 0.94 at 50 against a base rate of 0.200806, and 55 clients on both sides of the split. The jump from 0.85/0.64 to 0.95/0.94 is leakage, not improvement.

I also checked what happens to the grouped headline if I drop `measuredDayCount`, the feature I flagged in Methodology as tracking the same activity as the label. Precision at 20 and 50 fall to 0.05 and 0.10, which tells me the model leans on that one feature more than I'd like. As a check in the other direction, I deliberately let the model see `marchImpressions` itself, the same value the label is built from; precision at 20 and 50 both hit 1.00, exactly what I'd expect from a model that can see the answer, and that run was never a candidate for anything I'd submit.

Table 2 compares the logistic regression to the inverted-position baseline on that same grouped test set. The baseline's precision at 20 and 50 was 0.00 and 0.00; the logistic regression's was 0.85 and 0.64, both against the same base rate of 0.169252.

Table 1: Grouped holdout headline versus the leaky random-split comparison

Split	Precision@20	Precision@50	Base rate	Client overlap
Grouped client split (headline)	0.85	0.64	0.169252	0
Random page split (leaky, not trusted)	0.95	0.94	0.200806	55 clients

Table 2: Same grouped holdout: logistic regression versus the inverted-position baseline

Model	Precision@20	Precision@50	Base rate
Inverted-position baseline	0.00	0.00	0.169252
Logistic regression	0.85	0.64	0.169252

5 Limitations

This has been observational decision-support work, not an experiment. I didn't change anything on any of these pages, so I can't say that refreshing a page, rewriting it, or changing its word count would cause its visibility to go up; I can only say what was associated with what across March 2026. I'm also not claiming to have recovered or reverse-engineered Google's ranking algorithm. Five features and a logistic regression are a narrow lens, and a grouped precision at 20 of 0.85 doesn't mean I understand why Google ranks pages the way it does.

Table 1 and Table 2 are two views of one grouped client split: 20,632 scored test pages at a base rate of 0.169252. They are not two different client draws. A client-grouped shuffle with seed 42 still depends on the order unique client ids come back from the warehouse query, so an earlier Week 5 or Week 6 notebook run can print a different partition than this capstone re-run. I am reporting this re-run, not mixing it with those older splits.

The leakage checks in the Methodology section come from an association pass I ran early on: a linear regression built from five candidate features, evaluated in-sample, had a Spearman correlation of 0.8402 against March impressions. That number only covers the 110,403 pages with complete data on all five features, so it's a complete-case result, not a whole-dataset one, and I don't know whether the same relationship holds on the pages I had to leave out because a feature was missing.

The baseline's own top 20 is worth being skeptical of, too. Every page in it tied at a score of 1.0, and March impressions behind those rows ran from 1 to 4, not the kind of volume I'd want behind a ranking I was relying on.

None of this replaces an editor's judgment. Whatever score a page gets, someone still has to open it and look.

6 Ranked Recommendations

Here's what I'd actually tell a FlyRank editor to do with this, in order of how much I'd trust it.

First, watch the relationship between measured position and click-through rate, and don't expect word count to do much on its own. The CTR-by-position pattern held up: mean CTR was 0.003948 for positions 1 through 3 (n=27,174), 0.003034 for positions 4 through 10 (n=75,673), 0.002961 for positions 11 through 20 (n=29,606), and 0.001268 for position 21 and beyond (n=42,851). I'm calling that confirmed and directional, meaning it's a real pattern in the March data but not something I'd use to promise a specific CTR lift; 156,133 pages had no measured position at all and sit outside this comparison. Word count told a messier story. Pages with 501 to 1,500 words averaged about 122 impressions (n=69,252), pages with 1,501 to 3,000 words averaged about 1,527 (n=94,594), pages with 3,001 or more words averaged about 1,431 (n=60,095), and 107,430 pages had no word count recorded. That's not a line going up and to the right, and I left word count out of the baseline score for exactly that reason: an editor who reads this as "longer is always better" is reading it wrong.

Second, use the ranker's score, not the baseline's, to order the actual review queue, since it's the one with real precision behind it in the grouped holdout. The baseline's `assoc_position` reason code and its `watch_in_brief` action stay attached to each page as an explanation of why it showed up, but the ranking itself should come from the model. Cut the queue at the same top 20 and top 50 sizes I evaluated in the notebooks, since that's what the reported precision numbers describe.

Third, for tomorrow specifically: open the ranked list, review the highest-scoring pages first, and treat a page's score as a reason to look, not a reason to publish or reject anything automatically. Every page still needs a human editor to look at it.

I'd put my confidence in this at low to medium. A precision at 20 of 0.85 on the grouped headline still means 3 of the top 20 pages weren't actually in the top-visibility group, so an editor working straight down the list will hit real misses. The baseline's own top 20 is thin evidence, too: those pages all tied at a score of 1.0, and March impressions behind them ran from 1 to 4, not a stack of consistent data.

7 Reproducibility

Anyone who wants to check my numbers can start from a fresh clone. The repository is at <https://github.com/zainkhan006/FlyML>, and the only setup step is `pip install -r requirements.txt`.

Every modeling notebook in this project uses the same random seed, 42. I set it once and reused it everywhere a train/test split or a model fit needed one, so the baseline scores, the grouped splits, and the ranker results reported above all come from the same seed rather than a favorable draw.

The six notebooks behind this paper are:

- `w01_research_question.ipynb`
- `w02_ml_task_framing.ipynb`
- `w03_data_contract.ipynb`
- `w04_baseline_score.ipynb`
- `w05_model.ipynb`
- `w06_validation_audit.ipynb`
- `capstone.ipynb`

The ranking tables above were regenerated from `capstone.ipynb` on seed 42. Weeks 1 through 6 are the method trail. Because the grouped split shuffles client ids in whatever order the warehouse returns them, those older notebooks can print a different train/test partition than this paper even on the same seed.

Getting the paper in front of a reader is a separate step from getting the code in front of a reader. I compile `capstone.tex` in Overleaf to produce a PDF, then I put that PDF on GitHub Pages, for example as `docs/index.html` plus `docs/capstone.pdf`, or some equivalent layout that hosts the PDF at a stable HTTPS address. Whatever that address ends up being, it's the only thing that goes into `submission/paper_url.txt`: one line, one URL. I haven't written the Pages HTML yet; this section just describes the recipe.

8 Acknowledgments

This paper is built on the FlyRank ML Internship dataset, available at <https://flyrank.ai>. I'm grateful to the FlyRank ML Internship program for the access to real production data and for a structure that pushed me from a starter CSV toward an honest, grouped evaluation instead of a flattering one. Any mistakes left in this analysis are mine, not the program's.